LINGUAGEM DE PROGRAMAÇÃO I

EXCEÇÕES EM PYTHON

UNIVERSIDADE FEDERAL DO ACRE

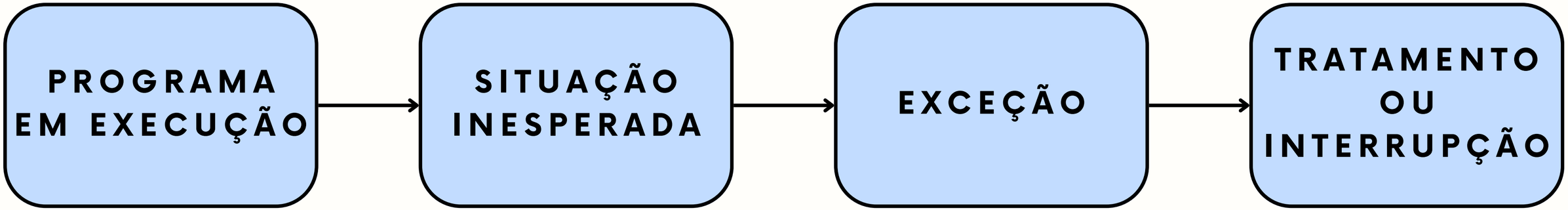
PROF.: CLECIO ELIAS

EMAIL.:CLECIO.SILVA@UFAC.BR

Exceções em Python

Definição

Exceções são **eventos** que ocorrem durante a execução de um programa e interrompem seu fluxo normal. Geralmente representam situações inesperadas ou erros que precisam ser identificados e tratados.



Exceções em Python

Erros de sintaxe e erros em execução

Erros podem ocorrer tanto na escrita do código quanto durante sua execução. Erros de sintaxe impedem a correta interpretação do programa, enquanto exceções surgem durante a execução de instruções sintaticamente válidas.

```
print("Olá"          # erro de sintaxe

10 / 0              # exceção
```

ERRO DE SINTAXE
→ ESTRUTURA INCORRETA DO CÓDIGO

EXCEÇÃO
→ ERRO OCORRIDO DURANTE A EXECUÇÃO

Exceções em Python

O que acontece quando ocorre uma exceção?

Quando não existe tratamento adequado, o programa apresenta informações sobre o erro por meio de um traceback e sua execução é interrompida.



Exceções em Python

Exemplos de exceções

Diferentes situações em tempo de execução originam diferentes tipos de exceções. Identificar seu tipo permite realizar tratamentos específicos.

```
20 * (2 / 0)
# ZeroDivisionError
```

```
5 + teste * 2
# NameError
```

```
"10" + 10
# TypeError
```

```
lista = [1, 2, 3]
lista[3]
# IndexError
```

Exceções em Python

Python possui uma **hierarquia** de classes de exceções. A classe Exception representa a base de diversas exceções utilizadas no tratamento de erros comuns.

Exceção	**Quando ocorre**	**Exemplo**
SyntaxError	Quando existe um erro na sintaxe do código	print("Olá"
KeyError	Ao acessar uma chave inexistente em um dicionário	d["cidade"]
TypeError	Quando uma operação é aplicada a um tipo inadequado	"10" + 10
ValueError	Quando o tipo está correto, mas o valor é inadequado	int("abc")
ZeroDivisionError	Quando ocorre uma divisão por zero	10 / 0
IndexError	Ao acessar uma posição inexistente de uma sequência	lista[10]
OSError	Em erros relacionados ao sistema ou operações de entrada/saída	Falha ao acessar arquivo/dispositivo

Exceções em Python

Estrutura try/except

O tratamento de exceções permite executar um conjunto de instruções e definir o que deve acontecer caso uma exceção seja produzida.

```
try:
    instruções

except:
    tratamento
```

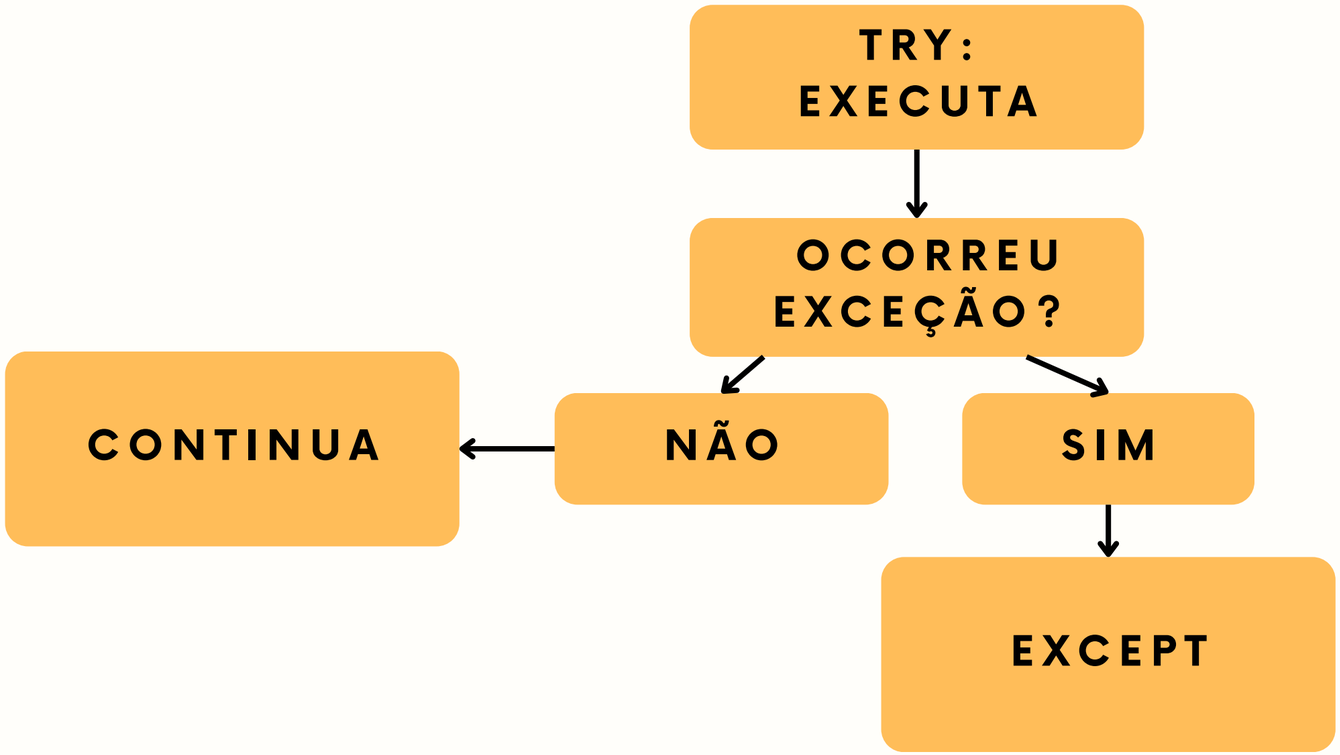
```
try:
    x = int(input("Entre com um número: "))
    print(x)

except:
    print("Valor inválido!")
```

Exceções em Python

Funcionamento do try/except

Primeiro, Python executa as instruções do bloco try. Caso nenhuma exceção ocorra, o bloco except é ignorado. Se ocorrer uma exceção prevista, a execução é direcionada para o tratamento correspondente.



Exceções em Python

Tratando uma exceção específica

É recomendado especificar o tipo de exceção que se deseja tratar. Dessa forma, o programa responde de maneira adequada a uma situação conhecida.

```
while True:
    try:
        x = int(input("Entre com um número: "))
        print(x)
        break

    except ValueError as erro:
        print(f"Erro: {erro}")
        print("Tente novamente!")
```

Exceções em Python

Múltiplas exceções

Um mesmo bloco try pode possuir diferentes cláusulas except, permitindo executar tratamentos distintos para diferentes tipos de exceção.

```
try:
    s = input("Entre com um número: ")
    x = 5 / int(s)

except ValueError:
    print("Número inválido!")

except ZeroDivisionError:
    print("Impossível dividir por zero!")

except EOFError:
    print("Programa interrompido!")
```

Exceções em Python

A cláusula else

A cláusula else é executada somente quando o bloco try termina sem produzir nenhuma exceção.

```
try:
    s = input("Entre com um número: ")
    x = 5 / int(s)

except ValueError:
    print("Número inválido!")

except ZeroDivisionError:
    print("Divisão por zero!")

else:
    print(f"Resultado: {x:.2f}")
```

Exceções em Python

A cláusula finally

O bloco finally é executado independentemente de uma exceção ter ocorrido ou não.

```
try:
    s = input("Entre com um número: ")
    x = int(s)

except ValueError:
    print("Número inválido!")

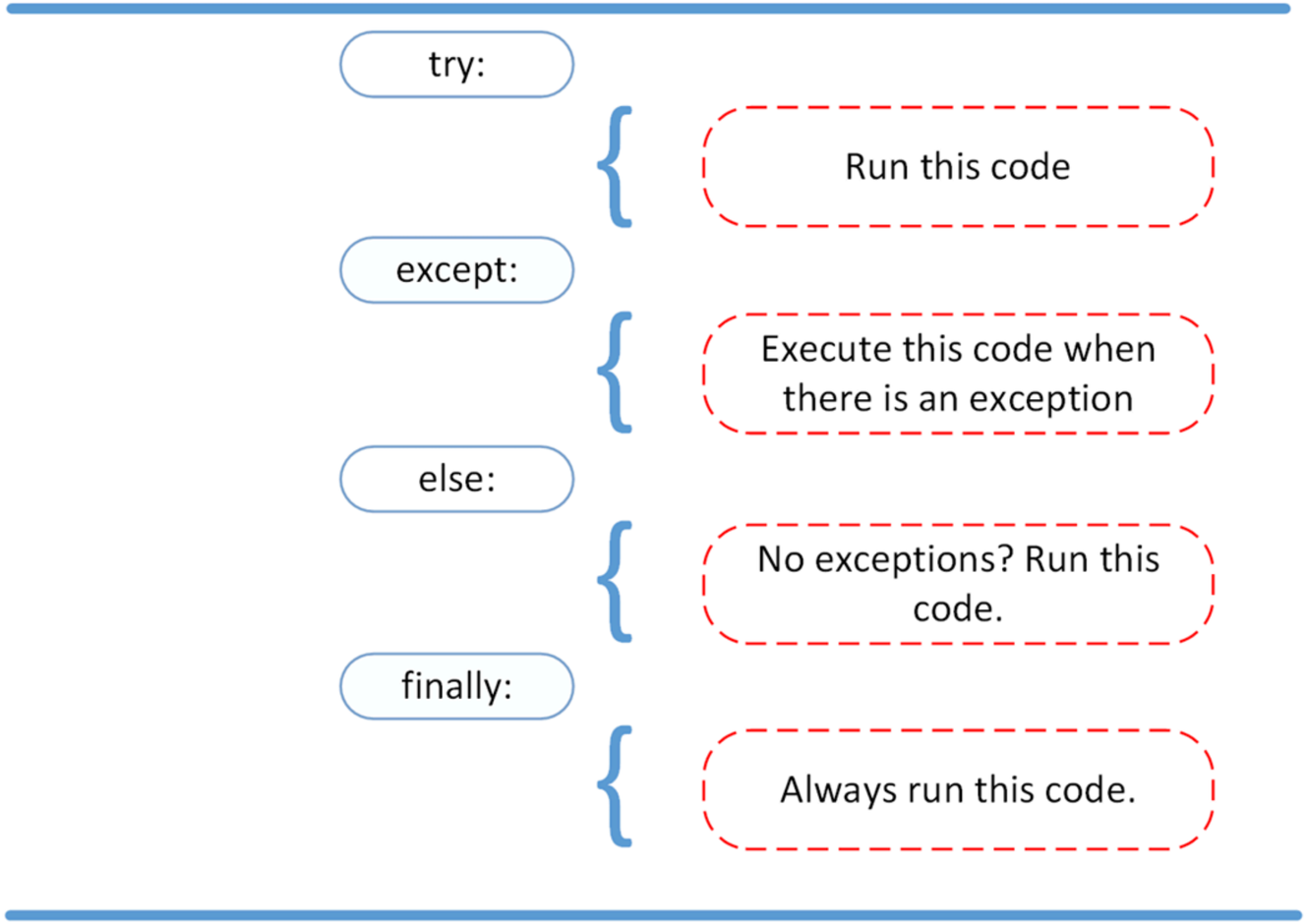
finally:
    print(f"O valor inserido foi: {s}")
```

TRY → TENTA EXECUTAR
EXCEPT → TRATA O ERRO
ELSE → EXECUTA SE NÃO HOUVER ERRO
FINALLY → EXECUTA SEMPRE

Exceções em Python

Resumo do tratamento de exceções

As cláusulas try, except, else e finally possuem responsabilidades diferentes e podem ser combinadas para controlar adequadamente situações excepcionais.



Exceções em Python

Lançando exceções com raise

A instrução raise permite gerar uma exceção explicitamente quando o programa identifica uma situação considerada inválida.

```
s = int(input("Entre com um número: "))


if s < 0:
    raise ValueError(
        "Número negativo não é permitido!"
    )
```

```
except ValueError as erro:
    print(erro)
```

Exceções em Python

Relançando uma exceção

Uma **exceção capturada** também pode ser lançada novamente utilizando raise. Isso permite executar alguma ação local e deixar que outra parte do programa realize o tratamento definitivo.

```
try:
    s = int(input("Número: "))

    if s < 0:
        raise ValueError("Número negativo!")

except ValueError:
    print("Número inválido!")
    raise
```

Exceções em Python

Criando exceções próprias

Python permite definir novos tipos de exceção por meio de classes derivadas de Exception. Por convenção, seus nomes normalmente terminam com Error.

```
class MinhaExcecaoError(Exception):
    pass


raise MinhaExcecaoError("Meu erro!")


except MinhaExcecaoError as erro:
    print(erro)
```

Exceções em Python

Boas práticas no uso de exceções

Exceções devem representar situações realmente excepcionais. Seu tratamento deve ser utilizado quando o programa sabe como responder adequadamente ao problema ocorrido.

- não utilizar exceções como fluxo normal do programa;
- capturar somente exceções que se sabe tratar;
- evitar excesso de blocos try/except;
- conhecer os tipos de exceção disponíveis;
- fornecer tratamentos adequados para cada situação.

Exceções em Python

Asserções em Python

Asserções são verificações utilizadas principalmente durante o desenvolvimento e a depuração de programas. Elas permitem confirmar se determinada condição considerada verdadeira pelo programador realmente está sendo satisfeita.

```
n = 10

assert n > 0

# se true: Programa continua normalmente
# se não: AssertionError
```

Exceções em Python

Sintaxe de assert

A instrução assert recebe uma expressão a ser verificada e pode incluir uma mensagem que descreva o problema identificado.

```
assert expressão, mensagem
```

```
n = -35


assert n > 0, \
    f"Número {n} deve ser maior que zero"
```

```python
assert 3 > 2

lista = [1, 2, 3, 4, 5]
assert 4 in lista

n = 35
assert isinstance(n, int)
```

Verifica se 3 é maior que 2, condição é verdadeira

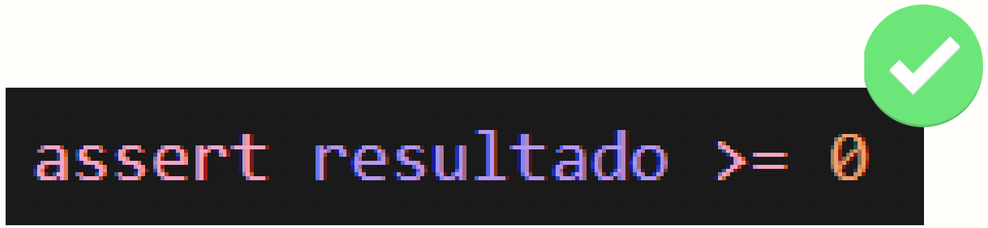
Verifica se o valor 4 está presente na lista. condição é verdadeira

Verifica se a variável n é do tipo inteiro (int). Como n = 35, a condição é verdadeira.

Exceções em Python

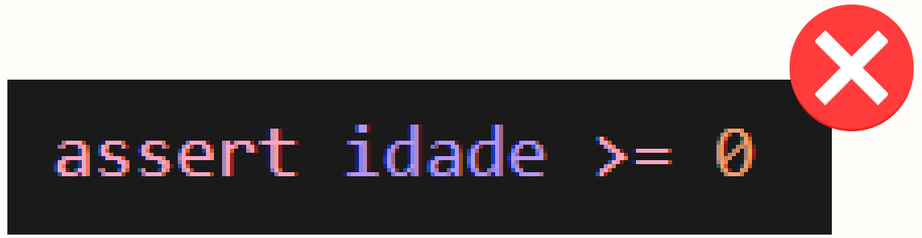
Quando usar e quando não usar assert

Asserções são apropriadas para detectar erros de programação durante o desenvolvimento, como verificação de pré-condições, pós-condições e testes. Elas não devem substituir o tratamento de dados inválidos em programas em produção.



```
assert resultado >= 0
```

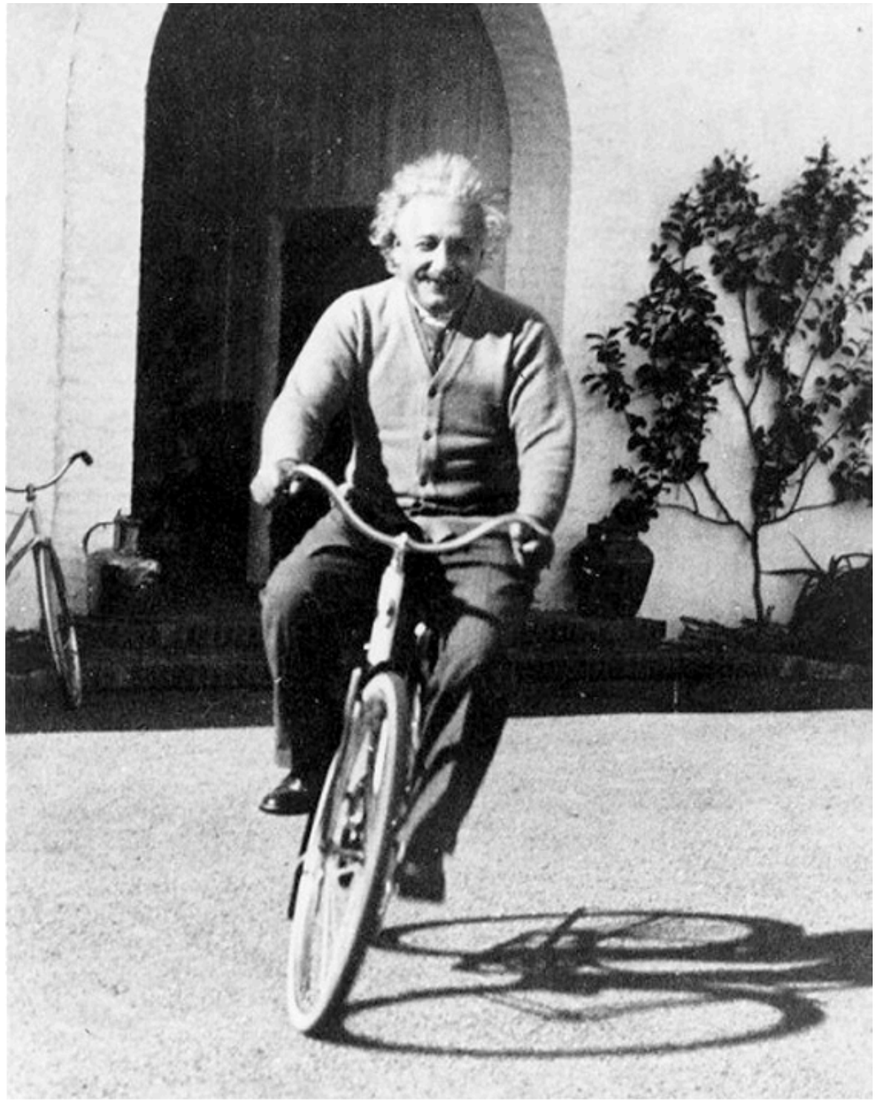
- depuração;
- testes;
- verificar condições internas;
- pré e pós-condições.



```
assert idade >= 0
```

- Não adequedo por exemplo para validar dados fornecidos pelo usuário.

Exercícios - URI



“A vida é como andar de bicicleta. Para manter o equilíbrio, é preciso se manter em movimento”.

- Einstein

Lista de exercicios!

Conteudos a seguir - programação

Strings + Lista exercicio (extra) Fecha N1	- 12/09 Sábado
Coleções (Tuplas e etc)	- 17/09 Quinta
Funções (+Func anônimas) + Lista exercicio	- 19/09 Sábado
List Comprehension	- 24/09 Quinta
Iteradores e func geradoras + Lista exercicio	- 26/09 Sábado
Exceções	- 08/10 Quinta
Extra (Prova Final caso necessario) + Lista exercicio	- 10/10 Sábado

Próximas listas (Nota completa da N2):
14/09 - 19/09
21/09 - 26/09
27/09 - 10/09